SISTEM PEMESANAN TIKET EVENT BERBASIS ARSITEKTUR MICROSERVICE

Untuk Mata Kuliah Pembangunan Perangkat Lunak Berorentasi Service

Disusun oleh

Muhammad Wahfiuddin

NIM 2410511091

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAKARTA

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan industri hiburan dan acara publik di Indonesia mendorong kebutuhan akan platform pemesanan tiket yang efisien, aman, dan dapat diskalakan. Sistem konvensional berbasis monolith sering kali kesulitan menangani lonjakan trafik saat penjualan tiket dibuka secara bersamaan, yang berujung pada downtime dan pengalaman pengguna yang buruk. Arsitektur berbasis microservices hadir sebagai solusi dengan memisahkan tanggung jawab domain bisnis ke dalam layanan-layanan independen yang dapat dikembangkan dan di-deploy secara terpisah. Sistem Pemesanan Tiket Event ini dibangun menggunakan pendekatan Service-Oriented Architecture (SOA) dengan tiga layanan utama, yakni Auth Service, Event Service, dan Order Service, yang diintegrasikan melalui sebuah API Gateway terpusat. Keamanan akses dijamin melalui mekanisme JWT dengan refresh token, token blacklisting saat logout, serta integrasi GitHub OAuth untuk autentikasi berbasis akun sosial. Pengelolaan tiket mencakup pembuatan kuota, proses checkout dengan pengecekan ketersediaan secara real-time, pembayaran, hingga penerbitan e-ticket dengan QR code unik per tiket. Sistem ini dirancang untuk memenuhi standar API modern dengan HTTP status code yang semantik, validasi input yang ketat, paginasi, dan filtering, sehingga siap digunakan dalam skenario produksi nyata.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan sebelumnya, adapun rumusan masalah dalam proyek ini antara lain:

1. Bagaimana merancang sistem pemesanan tiket event berbasis microservices yang memisahkan domain autentikasi, manajemen event, dan pemesanan ke dalam layanan independen?
2. Bagaimana mengimplementasikan mekanisme autentikasi yang aman menggunakan JWT dengan refresh token, token blacklisting, dan GitHub OAuth 2.0 Authorization Code Flow?
3. Bagaimana memastikan konsistensi data kuota tiket secara real-time ketika terjadi checkout bersamaan antar service yang berbeda?
4. Bagaimana menerbitkan e-ticket digital dengan QR code unik per tiket dan memvalidasi penggunaannya agar tidak dapat dipakai lebih dari satu kali?

1.3 Tujuan

Tujuan yang ingin dicapai dari proyek ini antara lain:

1. Membangun sistem pemesanan tiket event dengan arsitektur microservices yang terdiri dari Auth Service, Event Service, dan Order Service yang terintegrasi melalui API Gateway terpusat.

2. Mengimplementasikan sistem autentikasi berlapis menggunakan JWT, refresh token dengan masa berlaku terbatas, mekanisme blacklist saat logout, dan login via GitHub OAuth.
3. Menjamin keakuratan kuota tiket melalui inter-service call antara Order Service dan Event Service pada saat proses checkout berlangsung.
4. Menghasilkan e-ticket digital berupa QR code unik per tiket yang dapat divalidasi di pintu masuk dan ditandai sebagai sudah digunakan setelah scan pertama.

1.4 Ruang Lingkup

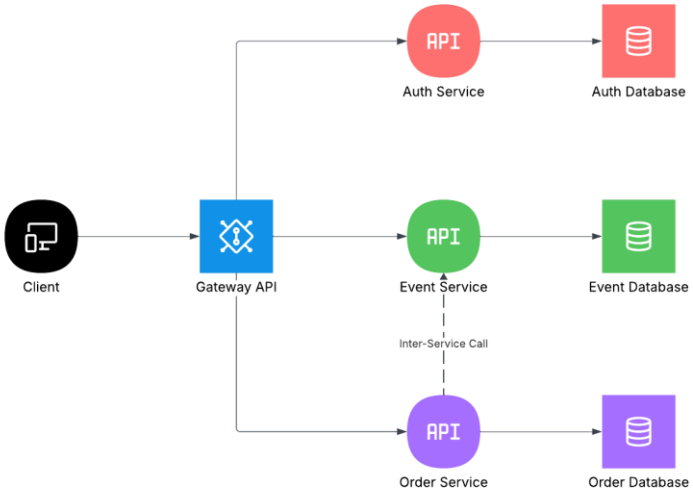
Ruang lingkup dalam proyek ini dibatasi sebagai berikut:

1. Sistem dibangun menggunakan Node.js/Express untuk API Gateway, Auth Service, dan Order Service, serta Laravel 11 untuk Event Service.
2. Autentikasi hanya mencakup GitHub OAuth sehingga provider lain tidak termasuk dalam scope pengerjaan.
3. Proses pembayaran bersifat simulasi (tidak terintegrasi dengan payment gateway nyata seperti Midtrans atau Xendit).
4. Sistem hanya mencakup tiga service utama, yakni autentikasi pengguna, manajemen event beserta tiketnya, dan pemesanan, sehingga fitur seperti notifikasi email, ulasan, dan rekomendasi event tidak termasuk.
5. Pengujian dilakukan menggunakan Postman collection sesuai skenario yang telah ditentukan, tanpa mencakup load testing atau pengujian performa.

BAB II

DESAIN SISTEM

2.1 Arsitektur Sistem



Sistem Pemesanan Tiket Event dibangun menggunakan pendekatan arsitektur microservices, di mana setiap domain bisnis dipisahkan ke dalam layanan independen yang memiliki tanggung jawab, basis data, dan siklus hidup pengembangannya masing-masing. Pendekatan ini dipilih untuk memastikan bahwa kegagalan pada satu layanan tidak berdampak langsung pada layanan lainnya, serta memungkinkan setiap service untuk dikembangkan dan di-deploy secara terpisah.

Seluruh request dari client masuk melalui satu titik masuk tunggal yaitu API Gateway yang berjalan pada port 3000. Gateway bertugas melakukan routing request ke service yang sesuai berdasarkan prefix path, memvalidasi JWT token untuk endpoint yang memerlukan autentikasi, serta menerapkan rate limiting untuk mencegah penyalahgunaan.

Sistem terdiri dari tiga layanan utama dengan peran sebagai berikut:

- Auth Service (Port 3001) yang dibangun menggunakan Node.js dan Express, bertanggung jawab atas seluruh proses autentikasi dan manajemen sesi pengguna, meliputi registrasi, login, penerbitan JWT access token dan refresh token, logout dengan mekanisme token blacklist, serta integrasi GitHub OAuth 2.0.
- Event Service (Port 3002) yang dibangun menggunakan PHP Laravel 11 dengan pola arsitektur MVC, bertanggung jawab atas manajemen data event dan kategori tiket, termasuk operasi CRUD event, pengelolaan kuota tiket, serta penyediaan listing event dengan dukungan pagination dan filter.
- Order Service (Port 3003) yang dibangun menggunakan Node.js dan Express, bertanggung jawab atas alur pemesanan dari checkout hingga penerbitan e-ticket, meliputi pembuatan order, simulasi pembayaran, generasi QR code unik per tiket, serta validasi tiket di pintu masuk.

Masing-masing service memiliki database MySQL tersendiri sehingga tidak ada ketergantungan langsung antar skema database. Komunikasi antar service yang diperlukan, seperti pengecekan dan pengurangan kuota tiket saat checkout, dilakukan melalui HTTP internal call antar service.

2.2 Desain API Gateway

API Gateway merupakan komponen sentral yang menjadi satu-satunya pintu masuk bagi seluruh request dari client. Dengan pola ini, client tidak perlu mengetahui alamat atau port masing-masing service secara langsung dan cukup berkomunikasi dengan Gateway pada port 3000, dan Gateway yang akan meneruskan request ke service yang tepat.

Mekanisme routing pada Gateway bekerja berdasarkan prefix path dari URL yang diminta. Setiap request yang masuk akan dicocokkan dengan aturan routing yang telah didefinisikan, kemudian diteruskan ke service tujuan menggunakan http-proxy-middleware. Tabel berikut merangkum pemetaan routing yang diterapkan pada sistem ini.

Method	Path	Service Tujuan
POST	/auth/register	Auth Service
POST	/auth/login	Auth Service
POST	/auth/refresh	Auth Service
POST	/auth/logout	Auth Service
GET	/auth/whoami	Auth Service
GET	/auth/github	Auth Service
GET	/auth/github/callback	Auth Service
GET	/events	Event Service
GET	/events/:id	Event Service
POST	/events	Event Service
PUT	/events/:id	Event Service
DELETE	/events/:id	Event Service
GET	/events/:id/tickets	Event Service
POST	/events/:id/tickets	Event Service
POST	/orders/checkout	Order Service
GET	/orders	Order Service
GET	/orders/:id	Order Service
POST	/orders/:id/pay	Order Service
GET	/orders/:id/ticket	Order Service
POST	/orders/validate-ticket	Order Service

Tidak semua endpoint memerlukan autentikasi. Gateway membedakan dua kategori endpoint: public yang dapat diakses tanpa token, dan protected yang mengharuskan client menyertakan JWT access token yang valid pada header Authorization: Bearer <token>. Middleware validasi JWT dijalankan sebelum request diteruskan ke service tujuan, sehingga service tidak perlu melakukan validasi token secara mandiri.

Apabila token tidak disertakan, token tidak valid, atau token telah masuk ke dalam blacklist akibat logout, Gateway akan langsung mengembalikan response 401 Unauthorized tanpa meneruskan request ke service manapun.

Endpoint yang bersifat public antara lain:

- POST /auth/register dan POST /auth/login, untuk proses autentikasi awal
- GET /auth/github dan GET /auth/github/callback, untuk alur GitHub OAuth
- GET /events dan GET /events/:id, untuk listing dan detail event yang dapat dilihat tanpa login

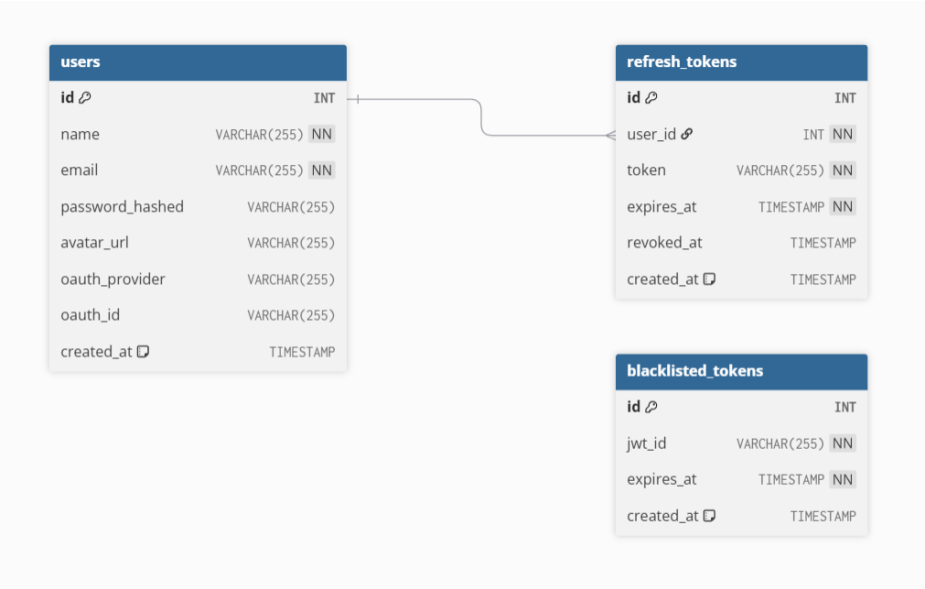
Seluruh endpoint di luar daftar tersebut memerlukan token yang valid.

Untuk mencegah penyalahgunaan API seperti brute force dan request flooding, Gateway menerapkan dua lapis rate limiting menggunakan express-rate-limit. Pertama, global limiter yang membatasi seluruh endpoint maksimal 20 request per menit per IP. Kedua, auth limiter yang khusus diterapkan pada endpoint /auth/login dan /auth/register dengan batas yang lebih ketat yaitu 10 request per 15 menit per IP, untuk mencegah serangan brute force pada proses autentikasi. Jika batas tersebut terlampaui, Gateway mengembalikan response 429 Too Many Requests.

2.3 Desain Database

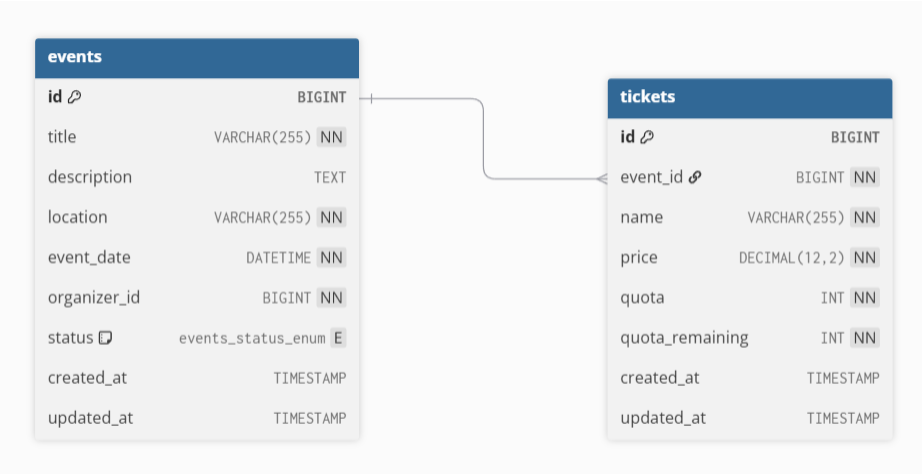
Sistem ini menerapkan pola Database per Service, di mana setiap service memiliki database MySQL tersendiri yang tidak dapat diakses langsung oleh service lain. Komunikasi antar service yang membutuhkan data lintas domain dilakukan melalui HTTP internal call, bukan melalui query langsung ke database milik service lain. Berikut adalah skema masing-masing database.

a. Auth Service Database (auth_db)
   Database ini dikelola oleh Auth Service dan menyimpan seluruh data yang berkaitan dengan identitas pengguna dan manajemen sesi.



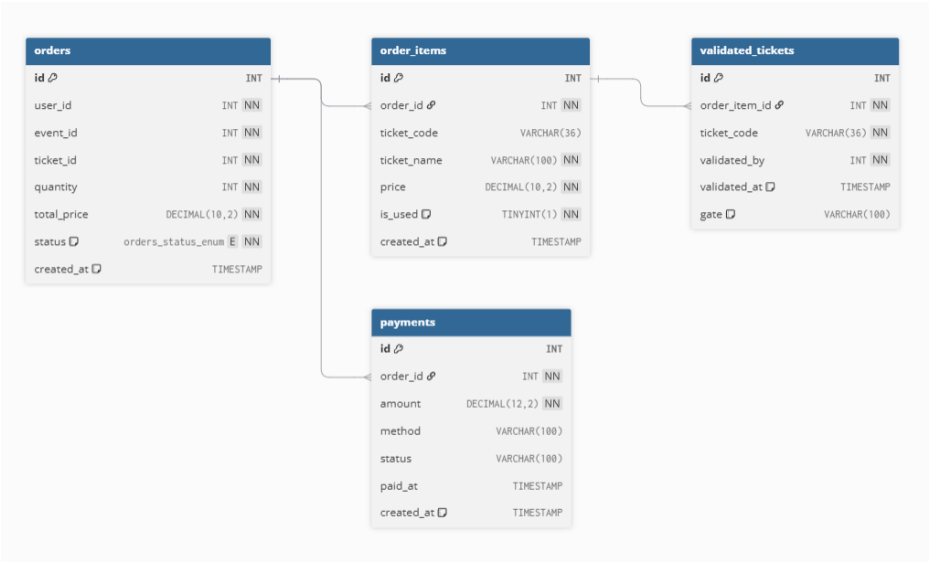
Tabel users menyimpan data pengguna baik yang mendaftar secara manual maupun melalui GitHub OAuth, dibedakan melalui kolom oauth_provider dan oauth_id. Kolom password_hashed bersifat nullable karena pengguna OAuth tidak memiliki password. Tabel refresh_tokens menyimpan hash SHA-256 dari refresh token dan bukan token aslinya untuk mencegah kebocoran data jika database diakses tanpa izin. Token yang sudah dipakai ditandai melalui kolom revoked_at. Tabel blacklisted_tokens menyimpan jwt_id dari access token yang sudah di-logout, digunakan oleh middleware checkBlacklist di Auth Service untuk memblokir token yang masih valid secara kriptografis namun sudah tidak sah secara sesi.

b. Event Service Database (event_db)
Database ini dikelola oleh Event Service berbasis Laravel 11 dan menyimpan data event beserta kategori tiketnya.



Tabel events menyimpan data event dengan kolom organizer_id yang merujuk ke ID pengguna di auth_db, namun tanpa foreign key lintas database. Meskipun begitu konsistensinya tetap dijaga di level aplikasi. Tabel tickets memiliki dua kolom kuota, yakni quota sebagai kuota awal yang tidak berubah, dan quota_remaining yang berkurang setiap kali ada checkout. Pengurangan kuota dilakukan menggunakan SELECT ... FOR UPDATE untuk mencegah race condition ketika terjadi pemesanan bersamaan.

c. Order Service Database (order_db)
Database ini dikelola oleh Order Service dan mencatat seluruh aktivitas transaksi dari checkout hingga validasi tiket di pintu masuk.



Tabel orders mencatat header transaksi dengan status pending, paid, atau cancelled. Tabel order_items menyimpan satu baris per tiket fisik yang artinya jika satu order membeli 3 tiket, akan ada 3 baris di tabel ini, masing-masing dengan ticket_code berupa UUID v4 yang unik dan menjadi dasar pembuatan QR code. Tabel payments mencatat transaksi pembayaran per order. Tabel validated_tickets mencatat log setiap kali tiket di-scan di pintu masuk, menyimpan siapa yang memvalidasi (validated_by) dan di gate mana.

2.4 Alur Autentikasi

Sistem autentikasi dirancang menggunakan dua mekanisme utama, yaitu autentikasi berbasis kredensial (email dan password) serta autentikasi OAuth 2.0 melalui GitHub. Seluruh alur autentikasi dikelola oleh Auth Service dan hanya dapat diakses melalui API Gateway.

a. Alur Registrasi dan Login

Pada proses registrasi, pengguna mengirimkan nama, email, dan password ke endpoint POST /auth/register. Auth Service memvalidasi kelengkapan input, memastikan email belum terdaftar, lalu menyimpan password dalam bentuk hash bcrypt. Setelah berhasil selanjutnya sistem menerbitkan sepasang token, yakni access token berupa JWT yang ditandatangani dengan algoritma HS256 dan memiliki masa berlaku singkat, serta refresh token berupa UUID v4 acak yang disimpan dalam bentuk hash SHA-256 di database. Kedua token tersebut dikembalikan ke client bersama data pengguna. Proses login mengikuti alur yang serupa. Client mengirimkan email dan password, Auth Service memverifikasi kecocokan password dengan hash yang tersimpan menggunakan bcrypt, lalu menerbitkan pasangan token baru.

b. Manajemen Token

Access token disisipkan pada setiap request ke endpoint yang dilindungi melalui header Authorization: Bearer <token>. Validasi token dilakukan di level API Gateway sebelum request diteruskan ke service tujuan. Apabila token valid, Gateway mengekstrak

user_id dan email dari payload JWT, lalu meneruskannya ke service melalui custom header X-User-Id dan X-User-Email.

Ketika access token habis masa berlakunya, client dapat memperbarui token tanpa harus login ulang dengan mengirimkan refresh token ke endpoint POST /auth/refresh. Auth Service memverifikasi keberadaan refresh token di database, memastikan token belum dicabut (kolom revoked_at bernilai null) dan belum kedaluwarsa, lalu menerbitkan pasangan token baru sekaligus mencabut refresh token lama.

c. Logout dan Token Blacklist

Proses logout dilakukan dengan mengirimkan access token dan refresh token ke endpoint POST /auth/logout. Auth Service memasukkan jwt_id dari access token ke tabel blacklisted_tokens, sehingga meskipun token masih valid secara kriptografis, token tersebut akan ditolak oleh middleware checkBlacklist pada setiap request selanjutnya ke endpoint yang memerlukan pengecekan blacklist. Refresh token yang dikirimkan juga langsung dicabut untuk mencegah pembaruan token setelah logout.

d. Autentikasi GitHub OAuth 2.0

Selain autentikasi berbasis kredensial, sistem mendukung login melalui akun GitHub menggunakan OAuth 2.0 Authorization Code Flow. Alurnya dimulai ketika client mengakses GET /auth/github, yang kemudian melakukan redirect ke halaman otorisasi GitHub dengan client_id, redirect_uri, dan scope user:email. Setelah pengguna memberikan izin, GitHub mengarahkan kembali ke GET /auth/github/callback beserta authorization code. Auth Service menukarkan code tersebut dengan GitHub access token, lalu menggunakannya untuk mengambil data profil pengguna dari GitHub API. Jika pengguna belum terdaftar, sistem membuatkan akun baru secara otomatis. Pada akhirnya, sistem menerbitkan pasangan access token dan refresh token yang sama seperti alur login biasa, sehingga client tidak perlu memperlakukan autentikasi OAuth secara berbeda dari sisi penanganan token.

2.5 Alur Bisnis Utama

Alur bisnis utama sistem mencakup tiga proses inti: pemesanan tiket, pembayaran, dan validasi e-ticket di pintu masuk

a. Checkout

Pengguna yang telah login mengirimkan event_id, ticket_id, dan quantity ke endpoint POST /orders/checkout. Order Service kemudian melakukan dua inter-service call ke Event Service: pertama untuk memverifikasi keberadaan event, kedua untuk mengambil detail tiket yang dipesan. Jika data valid, Order Service meminta pengurangan kuota tiket melalui PATCH /tickets/:id/reduce-quota. Event Service menjalankan pengurangan kuota menggunakan SELECT ... FOR UPDATE untuk mencegah race condition. Setelah kuota berhasil dikurangi, Order Service membuat record order dengan status pending dan menerbitkan sejumlah order item sesuai kuantitas, masing-masing dengan ticket_code berupa UUID v4 unik.

b. Pembayaran

Pengguna melakukan pembayaran dengan mengirimkan metode pembayaran ke POST /orders/:id/pay. Order Service memverifikasi bahwa order milik pengguna tersebut dan statusnya masih pending, lalu mencatat transaksi pembayaran dan mengubah status order menjadi paid. Pembayaran pada sistem ini bersifat simulasi tanpa integrasi payment gateway eksternal.

c. E-Ticket dan Validasi

Setelah order berstatus paid, pengguna dapat mengakses e-ticket melalui GET /orders/:id/ticket. Sistem mengembalikan daftar tiket beserta QR code dalam format data URL yang di-generate dari ticket_code masing-masing. Saat validasi di pintu masuk, petugas mengirimkan ticket_code ke POST /orders/validate-ticket. Sistem memeriksa keberadaan tiket, memastikan belum pernah digunakan (kolom is_used), lalu menandainya sebagai sudah digunakan dan mencatat log validasi ke tabel validated_tickets.

BAB III

IMPLEMENTASI SISTEM

3.1 Implementasi Arsitektur Sistem

Implementasi arsitektur sistem pada aplikasi pemesanan tiket event ini mengadopsi pendekatan microservices, di mana setiap domain bisnis dipisahkan ke dalam layanan yang berdiri sendiri, yaitu Auth Service yang berjalan di port 3001, Event Service yang berjalan di port 3002, dan Order Service yang berjalan di port 3003. Masing-masing service dijalankan pada port yang berbeda dan dikembangkan menggunakan teknologi yang sesuai dengan kebutuhannya, seperti Node.js dan Express untuk Auth dan Order Service, serta Laravel untuk Event Service. Seluruh request dari client tidak langsung menuju service, melainkan melalui satu pintu masuk terpusat yaitu API Gateway yang berjalan di port 3000, bertugas mengatur routing dan komunikasi antar service.

Dalam implementasinya, setiap service memiliki database MySQL tersendiri sesuai dengan prinsip database per service, sehingga tidak terjadi ketergantungan langsung antar database. Komunikasi antar service dilakukan melalui HTTP request internal, khususnya pada proses yang membutuhkan data lintas domain seperti pengecekan dan pengurangan kuota tiket saat checkout. Pendekatan ini memungkinkan sistem untuk lebih fleksibel, mudah dikembangkan, serta meningkatkan isolasi kesalahan, sehingga apabila terjadi gangguan pada satu service, service lain tetap dapat berjalan tanpa terdampak secara langsung.

3.2 Implementasi API Gateway

API Gateway diimplementasikan sebagai komponen pusat yang menjadi satu-satunya titik masuk bagi seluruh request dari client. Gateway dibangun menggunakan Node.js dan Express, serta memanfaatkan middleware seperti http-proxy-middleware untuk meneruskan request ke service tujuan berdasarkan prefix path yang diterima. Setiap endpoint seperti /auth, /events, dan /orders dipetakan ke service masing-masing, sehingga client tidak perlu mengetahui detail alamat atau port dari tiap service. Pendekatan ini menyederhanakan komunikasi sekaligus meningkatkan keamanan karena seluruh kontrol akses terpusat di satu layer.

Selain melakukan routing, API Gateway juga mengimplementasikan beberapa middleware penting, seperti validasi JWT untuk endpoint yang dilindungi serta rate limiting untuk mencegah penyalahgunaan API. Token yang dikirim melalui header Authorization akan diverifikasi sebelum request diteruskan ke service terkait, dan jika token tidak valid atau termasuk dalam blacklist, Gateway akan langsung mengembalikan response 401 Unauthorized. Selain itu, diterapkan pembatasan jumlah request baik secara global maupun khusus pada endpoint autentikasi untuk mengurangi risiko serangan seperti brute force dan request flooding.

3.3 Implementasi Microservice

3.3.1 Auth Service

Auth Service diimplementasikan menggunakan Node.js dan Express sebagai layanan yang bertanggung jawab penuh terhadap proses autentikasi dan manajemen sesi pengguna.

Dalam implementasinya, service ini menyediakan endpoint untuk registrasi, login, refresh token, logout, serta integrasi dengan GitHub OAuth. Setiap data pengguna yang masuk akan divalidasi terlebih dahulu, kemudian password disimpan dalam bentuk hash menggunakan bcrypt untuk menjaga keamanan data. Service ini juga menghasilkan access token berbasis JWT dan refresh token untuk menjaga keberlangsungan sesi tanpa harus melakukan login ulang secara berulang.

Selain itu, Auth Service mengelola penyimpanan refresh token dalam bentuk hash SHA-256 di database untuk menghindari penyalahgunaan jika terjadi kebocoran data. Mekanisme logout diimplementasikan dengan menambahkan identifier dari access token ke dalam daftar blacklist, sehingga token yang masih valid secara waktu tetap tidak dapat digunakan kembali.

3.3.2 Event Service

Event Service diimplementasikan menggunakan framework Laravel 11 untuk mengelola data event dan tiket. Service ini menyediakan berbagai endpoint untuk operasi CRUD event serta pengelolaan kategori tiket yang terkait dengan setiap event. Dalam implementasinya, setiap event memiliki relasi dengan beberapa jenis tiket yang memiliki atribut seperti harga, kuota awal, dan kuota tersisa. Validasi input dilakukan pada setiap request menggunakan request form untuk memastikan data yang disimpan tetap konsisten dan sesuai dengan aturan bisnis.

Salah satu bagian penting dalam implementasi Event Service adalah mekanisme pengelolaan kuota tiket. Untuk menghindari terjadinya race condition saat proses checkout berlangsung secara bersamaan, sistem menerapkan mekanisme transaksi database dengan penguncian baris data (row-level locking). Dengan cara ini, setiap proses pengurangan kuota akan dieksekusi secara aman dan terhindar dari kondisi inkonsistensi data. Event Service juga menyediakan endpoint yang dapat diakses oleh service lain secara internal, khususnya Order Service, untuk kebutuhan validasi dan sinkronisasi data tiket.

3.3.3 Order Service

Order Service diimplementasikan menggunakan Node.js dan Express untuk menangani seluruh alur pemesanan tiket, mulai dari proses checkout hingga validasi tiket di pintu masuk. Dalam implementasinya, service ini menerima permintaan checkout dari pengguna yang telah terautentikasi, kemudian melakukan komunikasi dengan Event Service untuk memverifikasi ketersediaan tiket dan mengurangi kuota secara real-time. Setelah proses tersebut berhasil, sistem akan membuat data order dengan status awal pending serta menghasilkan sejumlah item tiket sesuai dengan jumlah yang dipesan.

Setiap tiket yang dihasilkan memiliki kode unik berupa UUID v4 yang digunakan sebagai identitas e-ticket sekaligus dasar untuk pembuatan QR code. Setelah pengguna melakukan pembayaran, status order akan diperbarui menjadi paid, dan tiket dapat diakses dalam bentuk QR code. Pada tahap validasi, sistem akan memeriksa keabsahan tiket berdasarkan kode unik tersebut dan memastikan bahwa tiket belum pernah digunakan sebelumnya. Jika valid, tiket akan ditandai sebagai telah digunakan dan dicatat dalam log validasi.

3.4 Implementasi OAuth 2.0 (GitHub Authentication)

Selain autentikasi berbasis kredensial yang telah diimplementasikan pada Auth Service, sistem juga mendukung autentikasi menggunakan GitHub OAuth 2.0 dengan skema Authorization Code Flow. Dalam implementasinya, proses dimulai ketika client mengakses endpoint khusus yang akan mengarahkan pengguna ke halaman otorisasi GitHub. Setelah pengguna memberikan izin, GitHub akan mengembalikan authorization code ke endpoint callback yang telah didaftarkan pada sistem. Kode tersebut kemudian diproses oleh Auth Service untuk ditukarkan menjadi access token dari GitHub, yang selanjutnya digunakan untuk mengambil data profil pengguna seperti email, username, dan URL avatar milik user.

Data pengguna yang diperoleh dari GitHub akan digunakan untuk proses autentikasi di dalam sistem. Jika pengguna belum terdaftar, Auth Service secara otomatis membuat akun baru tanpa password dengan menandai sumber autentikasi sebagai OAuth. Jika pengguna sudah ada, sistem akan langsung melanjutkan proses login. Setelah proses ini selesai, sistem tetap menggunakan mekanisme yang sama seperti autentikasi biasa, yaitu menerbitkan access token berbasis JWT dan refresh token untuk digunakan oleh client. Integrasi OAuth ini tidak mengubah alur utama autentikasi, melainkan menjadi alternatif metode login yang lebih fleksibel dan aman bagi pengguna.

3.5 Implementasi Ticketing

Implementasi ticketing dalam sistem ini berfokus pada alur end-to-end pemesanan tiket, dimulai dari proses checkout hingga tiket digunakan di lokasi event. Setelah pengguna memilih event dan jenis tiket, sistem akan memproses permintaan checkout dengan memverifikasi ketersediaan tiket secara real-time melalui komunikasi dengan Event Service. Jika kuota mencukupi, sistem akan mengunci dan mengurangi jumlah tiket yang tersedia, kemudian membuat data pesanan beserta detail tiket yang dipesan. Setiap tiket direpresentasikan sebagai entitas unik untuk memastikan bahwa setiap pembelian dapat dilacak dan dikelola secara individual.

Setelah proses pembayaran berhasil, sistem akan menghasilkan e-ticket dalam bentuk QR code yang merepresentasikan kode unik dari setiap tiket. QR code ini dapat diakses oleh pengguna dan digunakan saat memasuki area event. Pada tahap validasi, sistem akan memproses kode tiket yang di-scan untuk memastikan bahwa tiket tersebut valid dan belum pernah digunakan sebelumnya. Jika valid, tiket akan ditandai sebagai telah digunakan dan dicatat dalam sistem untuk mencegah penggunaan ulang.

BAB IV

PENGUJIAN SISTEM

4.1 Metode Pengujian

Pengujian sistem pada aplikasi pemesanan tiket event ini dilakukan menggunakan pendekatan black-box testing, yaitu metode pengujian yang berfokus pada fungsionalitas sistem tanpa melihat implementasi kode di dalamnya. Pengujian dilakukan dengan mengirimkan berbagai request ke endpoint API dan mengamati apakah output yang dihasilkan telah sesuai dengan spesifikasi yang dirancang pada tahap desain. Tools yang digunakan dalam proses ini adalah Postman, yang memudahkan dalam pengelolaan dan pengujian berbagai skenario request secara terstruktur.

Ruang lingkup pengujian mencakup seluruh fitur utama sistem, yaitu autentikasi pengguna, manajemen event, serta proses pemesanan dan validasi tiket. Setiap endpoint diuji menggunakan beberapa skenario, baik skenario normal (valid input) maupun skenario error (invalid input atau kondisi tertentu), untuk memastikan sistem mampu menangani berbagai kemungkinan yang terjadi. Hasil pengujian kemudian dibandingkan dengan output yang diharapkan untuk menentukan apakah fungsi sistem berjalan dengan benar sesuai dengan kebutuhan yang telah ditetapkan.

4.2 Skenario Pengujian

4.2.1 Auth Service

Kode	Skenario	Endpoint	Expected Result	Hasil
AT-1-s	Registrasi berhasil dengan field lengkap (name, email, password)	POST /auth/register	201 Created dengan menghasilkan access_token dan refresh token, serta user baru berhasil disimpan di DB	PASS
AT-1-e1	Registrasi gagal dengan email yang sudah terdaftar	POST /auth/register	409 Conflict dengan message “Email already exists”	PASS
AT-1-e2	Registrasi gagal dengan field tidak lengkap	POST /auth/register	400 Bad Request dengan message “Missing required fields!”	PASS
AT-2-s1	Login berhasil dengan email dan password yang benar dan terdaftar	POST /auth/login	200 OK dengan return access_token dan refresh token, serta data user yang sama	PASS
AT-2-e1	Login gagal dengan password yang salah	POST /auth/login	401 Unauthorized dengan message “Invalid credentials”	PASS

AT-2-e2	Login gagal dengan field kosong	POST /auth/refresh	400 Bad Request dengan message “Missing required fields!”	PASS
AT-3-s1	Login dengan GitHub	GET /auth/github	302 Redirect ke halaman login/authorization GitHub	PASS
AT-3-s2	Login dengan GitHub berhasil dan user telah terdaftar	GET /auth/github/callback	201 Created dengan menghasilkan access_token dan refresh token, serta user baru berhasil disimpan di DB	PASS
AT-4-s	Berhasil cek data user yang memiliki access token saat ini	GET /auth/whoami	200 OK dengan return id user, name, email, avatar_url, dan oauth_provider	PASS
AT-4-e1	Gagal cek data user yang tanpa access token	GET /auth/whoami	401 Unauthorized dengan message “Unauthorized”	PASS
AT-5-s	Refresh token baru berhasil	POST /auth/refresh	200 OK menghasilkan access token baru yang valid	PASS
AT-5-e	Refresh token baru gagal dengan refresh token yang tidak valid	POST /auth/refresh	401 Unauthorized dengan message "Invalid refresh token"	PASS
AT-6-s	Logout berhasil	POST /auth/logout	200 OK dengan message “Logout success” dan token masuk ke table blacklisted_tokens	PASS
AT-4-e2	Gagal cek data user dengan token yang sudah blacklisted	GET /auth/whoami	401 Unauthorized dengan message “Token has been revoked”	PASS
AT-2-s2	Login berhasil setelah logout	POST /auth/login	200 OK dengan return access_token dan refresh token yang baru, serta data user yang sama	PASS

4.2.2 Event Service

Kode	Skenario	Endpoint	Expected Result	Hasil
ET-1-s	Event berhasil dibuat dengan field wajib terisi semua	POST /events	201 Created dengan return data event dan berhasil tersimpan di DB	PASS

ET-1-e1	Event gagal dibuat karena tanpa token	POST /events	401 Unauthorized dengan message “Unauthorized”	PASS
ET-1-e2	Event gagal dibuat dengan field tidak lengkap	POST /events	422 Unprocessable Content dengan message “Validation failed”	PASS
ET-2-s1	Berhasil menampilkan semua event	GET /events	200 OK berhasil menampilkan semua event dengan pagination	PASS
ET-2-s2	Berhasil menampilkan event dengan filter lokasi	GET /events?location=	200 OK berhasil menampilkan semua event sesuai dengan filter lokasi	PASS
ET-2-s3	Berhasil menampilkan event dengan filter status	GET /events?status=	200 OK berhasil menampilkan semua event sesuai dengan filter status	PASS
ET-2-s4	Berhasil menampilkan event dengan sistem pagination yang sesuai	GET /events?page=[x]&per_page=[y]	200 OK berhasil menampilkan semua event sesuai dengan pagination	PASS
ET-2-s5	Berhasil menampilkan event yang sesuai dengan ID-nya	GET /events/:id	200 OK dengan return data event	PASS
ET-2-e1	Gagal menemukan event sesuai dengan id	GET /events/:id	404 Not Found dengan message “Event not found”	PASS
ET-3-s	Berhasil memperbarui data event	PUT /events/:id	200 OK dengan return data event yang baru dan berhasil tersimpan di DB	PASS
ET-3-e1	Gagal memperbarui data event karena bukan pemilik event	PUT /events/:id	403 Forbidden dengan message “Forbidden”	PASS
ET-4-s	Berhasil membuat tiket	POST /events/:id/tickets	201 Created dengan return data tiket dan berhasil tersimpan di DB	PASS
ET-4-e1	Gagal membuat tiket karena bukan pemilik event	POST /events/:id/tickets	403 Forbidden dengan message “Forbidden”	PASS
ET-4-e2	Gagal membuat tiket karena harga yang tidak valid	POST /events/:id/tickets	422 Unprocessable Content dengan message “Validation failed”	PASS
ET-5-s	Berhasil menampilkan tiket berdasarkan event	GET /events/:id/tickets	200 OK dengan return data tiket	PASS

ET-5-e	Gagal menampilkan tiket berdasarkan karena event tidak ditemukan	GET /events/:id/tickets	200 OK dengan array yang kosong	PASS
ET-6-s	Berhasil menghapus event	DELETE /events/:id	204 No Content	PASS
ET-6-e1	Gagal menghapus event karena bukan pemilik event	DELETE /events/:id	403 Forbidden dengan message “Forbidden”	PASS
ET-6-e2	Gagal menghapus event karena event tidak ditemukan	DELETE /events/:id	404 Not Found dengan message “Event not found”	PASS

4.2.3 Order Service

Kode	Skenario	Endpoint	Expected Result	Hasil
OT-1-s	Berhasil checkout order dengan semua field lengkap dan kuota mencukupi	POST /orders/checkout	201 Created dengan ticket code sebanyak quantity dan data telah tesimpan di DB serta kuota tiket berkurang di event_db	PASS
OT-1-e1	Gagal checkout order karena kuota tidak mencukupi	POST /orders/checkout	409 Conflict dengan message “Invalid quantity to complete order”	PASS
OT-1-e2	Gagal checkout order karena ID tiket tidak ditemukan	POST /orders/checkout	404 Not Found dengan message “Ticket not found”	PASS
OT-1-e3	Gagal checkout order karena ID event tidak ditemukan	POST /orders/checkout	404 Not Found dengan message “Event not found”	PASS
OT-1-e4	Gagal checkout order karena tanpa token	POST /orders/checkout	401 Unauthorized dengan message “Unauthorized”	PASS
OT-2-s1	Berhasil menampilkan semua order yang dimiliki user	GET /orders	200 OK dengan return semua data order milik user	PASS
OT-2-s2	Berhasil menampilkan order yang sesuai dengan ID order	GET /orders/:id	200 OK dengan return data order yang sesuai dengan ID order	PASS
OT-2-e1	Gagal menampilkan order yang sesuai	GET /orders/:id	403 Forbidden dengan message “Forbidden”	PASS

	karena bukan pemilik order			
OT-2-e2	Gagal menampilkan order karena order tidak ditemukan	GET /orders/:id	200 OK dengan array yang kosong	PASS
OT-3-s	Berhasil melakukan pembayaran order	POST /orders/:id/pay	201 Created dengan message “Payment success” dan status order berubah menjadi “paid”	PASS
OT-3-e1	Gagal melakukan pembayaran order karena bukan pemilik order	POST /orders/:id/pay	403 Forbidden dengan message “Forbidden”	PASS
OT-3-e2	Gagal melakukan pembayaran order karena sudah dibayar	POST /orders/:id/pay	400 Bad Request dengan message “Order is already paid”	PASS
OT-4-s	Berhasil menerbitkan e-ticket	GET /orders/:id/ticket	200 OK dengan return ticket code, ticket name, dan qr dalam bentuk data:image/png ;base64	PASS
OT-4-e1	Gagal menerbitkan e-ticket karena order belum dibayar	GET /orders/:id/ticket	400 Bad Request dengan message “Order is not paid”	PASS
OT-4-e2	Gagal menerbitkan e-ticket karena bukan pemilik order	GET /orders/:id/ticket	403 Forbidden dengan message “Forbidden”	PASS
OT-5-s	Berhasil melakukan validasi tiket	POST /orders/validate-ticket	200 OK dengan return valid = true dan message “Ticket validated”	PASS
OT-5 e1	Gagal melakukan validasi tiket karena tiket telah digunakan	POST /orders/validate-ticket	400 Bad Request dengan message “Ticket is already used”	PASS
OT-5 e2	Gagal melakukan validasi tiket karena tiket tidak ditemukan	POST /orders/validate-ticket	404 Not Found dengan message “Ticket not found”	PASS

BAB V

KESIMPULAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, sistem pemesanan tiket event berbasis arsitektur microservices ini berhasil dibangun sesuai dengan tujuan yang telah ditetapkan. Pemisahan layanan ke dalam Auth Service, Event Service, dan Order Service memungkinkan sistem memiliki struktur yang lebih modular, mudah dikembangkan, serta mampu mengisolasi kesalahan pada masing-masing layanan. Implementasi API Gateway sebagai pintu masuk terpusat juga terbukti efektif dalam mengatur routing, autentikasi, dan pembatasan akses ke seluruh service.

Dari sisi fungsionalitas, sistem telah mampu menangani proses utama seperti registrasi dan login pengguna, pengelolaan event dan tiket, proses checkout, pembayaran, hingga penerbitan dan validasi e-ticket berbasis QR code. Mekanisme autentikasi menggunakan JWT dan refresh token, serta integrasi GitHub OAuth, berjalan dengan baik dalam menjaga keamanan dan fleksibilitas akses pengguna. Berdasarkan hasil pengujian menggunakan Postman, seluruh endpoint utama menunjukkan hasil yang sesuai dengan spesifikasi yang dirancang, sehingga sistem dapat dikatakan telah memenuhi kebutuhan dasar sebagai platform pemesanan tiket event.

5.2 Saran

Untuk pengembangan lebih lanjut, sistem ini masih dapat ditingkatkan dari berbagai aspek. Salah satunya adalah integrasi dengan payment gateway nyata seperti Midtrans atau Xendit agar proses pembayaran tidak lagi bersifat simulasi dan lebih mendekati kondisi produksi. Selain itu, pengujian yang dilakukan saat ini masih terbatas pada pengujian fungsional, sehingga diperlukan pengujian tambahan seperti load testing dan stress testing untuk mengetahui performa sistem dalam menangani trafik tinggi.

Dari sisi fitur, sistem dapat dikembangkan dengan menambahkan notifikasi email atau notifikasi real-time kepada pengguna terkait status pesanan dan tiket. Penggunaan caching dan message broker juga dapat dipertimbangkan untuk meningkatkan performa dan skalabilitas sistem, terutama dalam komunikasi antar service. Terakhir, aspek keamanan dapat terus ditingkatkan, misalnya dengan penerapan rate limiting yang lebih adaptif, monitoring sistem, serta logging yang lebih komprehensif untuk mendukung proses audit dan debugging.